

MAC0425/MAC5739 - Inteligência Artificial (2026)

Quarto Exercício-Programa Planejamento Probabilístico e Aprendizado por Reforço

1 Introdução

O objetivo deste exercício-programa será implementar os algoritmos de Iteração de Valor (*Value Iteration* - *VI*) e Q-Learning. Para isso, utilizaremos parte do material/código livremente disponível do curso UC Berkeley CS188 ¹.

Testaremos o algoritmo VI e Q-Learning no domínio *Gridworld* (descrito a seguir). O algoritmo Q-Learning também será testado no domínio do Pac-Man, já apresentado nos EPs anteriores.

O ambiente *Gridworld*. A Figura 1 ilustra o ambiente *Gridworld* no qual um agente se move entre as células de uma matriz $n \times m$ nas direções Norte, Oeste, Sul e Leste (quando o caminho não estiver bloqueado). Dentro da matriz, algumas localizações são consideradas terminais: quando alcançadas o agente recebe uma recompensa específica e o episódio termina. A estocasticidade do ambiente *Gridworld* é definida da seguinte forma: quando o agente escolhe ir para o Norte ou para o Sul, ele tem 80% de chance de ir para a direção desejada; 10% de chance de ir para Oeste, e 10% de chance de ir para Leste. Analogamente, se o agente escolher ir para Leste ou Oeste, ele tem 80% de chance de ir para a direção desejada, 10% de chance de ir para Norte, e 10% de chance de ir para Sul.

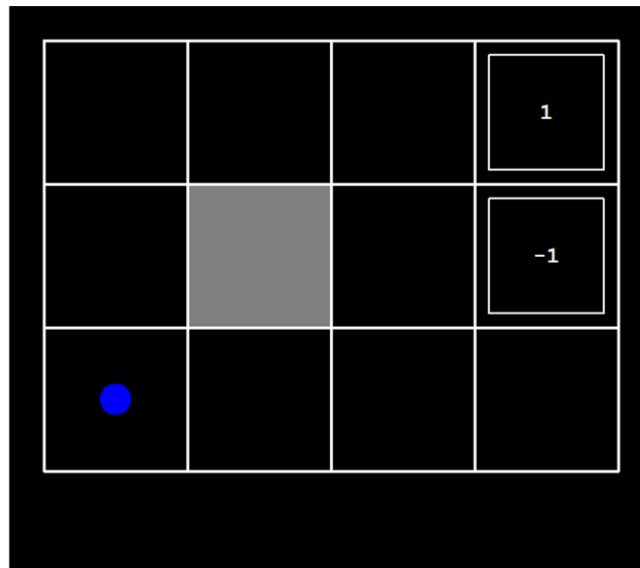


Figura 1: Exemplo de jogo do domínio *Gridworld*. O mapa é de tamanho 3×4 . O agente é representado pelo círculo azul. O quadrado cinza e as bordas são obstáculos que o agente não consegue atravessar. Os quadrados duplos são localizações terminais, e o número no centro de uma localização é a recompensa recebida pelo agente ao alcançá-la.

¹<https://inst.eecs.berkeley.edu/~cs188/archive/fa25/>

1.1 Instalação

Para a realização deste EP será necessário instalar em sua máquina a versão 3.9 do Python ². Será também necessário o pacote *tkinter*. Verifique se ele já está instalado usando o comando:

```
$ python -m tkinter
```

Caso não esteja, instale-o no seu sistema separadamente ou modifique sua instalação do Python para incluí-lo.

Faça o download do código novo no **arquivo ep4.zip** disponível na página da disciplina no e-Disciplinas. Descompacte o arquivo ep4.zip e rode na raiz do diretório o seguinte comando para testar a instalação:

```
$ cd reinforcement/  
$ python gridworld.py -m
```

Após executar esta linha de comando, uma nova janela deve ter sido aberta em seu desktop com o simulador sendo executado com o mapa padrão (1). Você pode mover o agente (representado pelo ponto azul) com as setas do teclado. Lembre que toda ação só tem 80% de chance de alcançar o resultado esperado.

2 O que você deve implementar?

Nesse exercício-programa, você deverá implementar várias funções relacionadas ao algoritmo Iteração de Valor e Q-learning, sinalizadas pelo comentário "*** ADD YOUR CODE HERE ***", nos arquivos **valueIterationAgents.py** e **qlearningAgents.py**.

2.1 Iteração de Valor

Na implementação de **Iteração de Valor**, você deverá usar alguns métodos e atributos do objeto MDP recebido no construtor da classe **ValueIterationAgent**. São esses:

- **self.mdp.getStates()**: Devolve um conjunto com todos os estados do problema (conjunto $\mathcal{S}$).
- **self.mdp.getPossibleActions(state)**: Devolve uma lista de ações possíveis no dado estado (conjunto $\mathcal{A}$ já filtrado por estado).
- **self.mdp.getTransitionStatesAndProbs(state, action)**: Devolve uma lista de pares $\langle \text{nextState}, \text{prob} \rangle$ representando a devolução da função de transição $\mathcal{T}$. Isto é, a probabilidade de transição para o próximo estado **nextState** ao executar a ação **action** no estado atual **state**, dada por **prob**.
- **self.mdp.getReward(state, action, nextState)**: Devolve a recompensa imediata obtida após executar a ação **action** no estado **state** e transicionar para o estado **nextState** (função r para o **state**).

²<https://www.python.org/downloads/>

- `self.mdp.isTerminal(state)`: Devolve se o estado é terminal e portanto não há mais recompensas a serem obtidas.
- `self.discount`: Fator de desconto do MDP (γ).
- `self.iterations`: Atributo indicando o limite de iterações do algoritmo.

Note que a função `getReward` acima recebe três argumentos, $(state, action, nextState)$, diferentemente do algoritmo descrito nos slides da aula 18 em que a recompensa é definida como $r(s)$. Por conta disto, o operador de atualização de Bellman que você deverá implementar precisa ser adaptado para:

$$Q(s, a) \leftarrow \sum_{s' \in S} \mathcal{T}(s, a, s') * (r(s, a, s') + \gamma * V(s')).$$

2.2 Q-Learning

Na implementação do **Q-Learning** (na classe `QLearningAgent`), o algoritmo não tem acesso ao modelo do MDP. Por isso, os métodos da classe são chamados externamente pelo simulador. O único método pré-implementado que você precisará chamar explicitamente é o `self.getLegalActions(state)`, implementado na superclasse `ReinforcementAgent`, que devolve as ações disponíveis para o estado `state`.

Além do fator de desconto `self.discount`, a classe do Q-Learning também possui os atributos `self.epsilon` (taxa de exploração ϵ) e `self.alpha` (taxa de aprendizado α), que também devem ser usados na implementação. Por padrão, os valores de α e ϵ são 0.2 e 0.05, respectivamente.

3 Experimentos

Código 1 - Iteração de Valor

Para implementar o algoritmo de Iteração de Valor você deverá completar a implementação da classe `ValueIterationAgent` no arquivo `valueIterationAgents.py`. Para isso, é necessário implementar três métodos:

- `computeQValueFromValues(self, state, action)`: um método que recebe um estado s e uma ação a , e devolve o valor de $Q(s, a)$ usando a função V embutida em `self.values`.
- `computeActionFromValues(self, state)`: um método que recebe um estado s , e usa a função V em `self.values` para determinar qual a melhor ação para ser escolhida quando no estado s . Em caso de empate, qualquer ação dentre as com o maior valor de $Q(s, a)$ pode ser devolvida. Note que no caso em que não tem ações possíveis no estado s , este método deve devolver `None`.
- `runValueIteration(self)`: a função principal da classe que precisa rodar o algoritmo Iteração de Valor. O valor inicial de $V(s)$ é zero para todos os estados. Este método não devolve nenhum valor. Apenas é necessário modificar `self.values` com os valores de V após a Iteração de Valor. **IMPORTANTE:** A computação de V_k em uma iteração deve usar apenas os valores de V_{k-1} , ou seja, dado um estado s e um de seus estados sucessores s' , para calcular $V_k(s)$ este método deve usar $V_{k-1}(s')$, mesmo se o valor de $V_k(s')$ já tenha sido calculado durante a mesma iteração.

Defina funções auxiliares se precisar. Para testar seu algoritmo no mapa da Figura 1 rode o comando:

```
$ python gridworld.py -a value -i 100 -k 10
```

Ao rodar o comando anterior, primeiro será mostrado uma tela com os valores computados de V para todos os estados (células) após 100 iterações (argumento `-i`) do algoritmo. Ao apertar qualquer tecla, será mostrado todos os valores da função Q baseados na função V . Ao apertar qualquer tecla novamente, serão simulados 10 episódios (argumento `-k`) usando a política gulosa derivada da função V . Para testar valores diferentes de α e ϵ , adicione o argumento `-a alpha=[taxa de aprendizado],epsilon=[taxa de exploração]`.

Teste também com valores diferentes para o fator de desconto (usando o parâmetro `--discount`) e/ou diferentes chances de ir para a direção errada (com `--noise`). Por exemplo, com os comandos:

```
$ python gridworld.py -a value -i 100 -g BridgeGrid --discount 1 --noise 0.2
$ python gridworld.py -a value -i 100 -g BridgeGrid --discount 0.9 --noise 0
```

No cenário `DiscountGrid`, você também pode especificar um valor para a recompensa imediata por cada ação realizada (argumento `--livingReward`). Teste diferentes parâmetros com o comando:

```
$ python gridworld.py -g DiscountGrid -a value \
  --discount [fator de desconto] \
  --noise [chance de erro] \
  --livingReward [recompensa por ação]
```

Código 2 - Q-Learning

Você deverá completar a implementação da classe `QLearningAgent`, no arquivo `qlearningAgents.py`. Para isso, é necessário implementar cinco métodos da classe:

- `getQValue(self, state, action)`: recebe um estado s e uma ação a , e devolve o valor atual de $Q(s, a)$. Se o par (s, a) nunca foi usado antes, deve ser devolvido 0.
- `computeValueFromQValues(self, state)`: recebe um estado s , e devolve o valor de $V(s)$ usando a função Q . Caso s seja um estado terminal, ou não houver ações possíveis no estado s , deve ser devolvido 0.
- `computeActionFromQValues(self, state)`: recebe um estado s , e devolve a melhor ação em s usando a função Q . É recomendado que empates sejam quebrados aleatoriamente. Se não houver nenhuma ação possível no estado s , deve ser devolvido `None`.
- `getAction(self, state)`: recebe um estado s , e devolve uma ação usando a política ϵ -gulosa. Ou seja, com probabilidade `self.epsilon`, escolha uma ação qualquer uniformemente entre as possíveis no estado s , e com probabilidade `1 - self.epsilon` devolva a ação gulosa no estado s . Caso não haja nenhuma ação possível no estado s , devolva `None`. **Obs:** para este método, pode ser útil chamar `util.flipCoin(p)`, que devolve `True` com probabilidade p e `False` com probabilidade $1 - p$. Também pode ser útil chamar `random.choice(list)` que devolve um elemento de uma lista `list` aleatoriamente com probabilidade uniforme.
- `update(self, state, action, nextState, reward)`: recebe um estado s , ação a , próximo estado s' e recompensa r . Este método faz uma atualização da função Q usando o algoritmo Q-Learning. Este método não devolve nenhum valor.

Para testar seu algoritmo, rode o comando:

```
$ python gridworld.py -a q -k 100
```

Os episódios executados, cuja quantidade é indicada pelo argumento `-k`, serão usados para treinamento do Q-Learning. Todos serão mostrados na interface gráfica.

Teste também com diferentes valores para a taxa de exploração (ϵ) com o argumento `-e`. Por exemplo:

```
$ python gridworld.py -a q -k 100 --noise 0.0 -e 0.1
$ python gridworld.py -a q -k 100 --noise 0.0 -e 0.9
```

Uma vez funcionando para o Gridworld, teste também o seu algoritmo no Pac-Man:

```
$ python pacman.py -p PacmanQAgent -x 2000 -n 2010 -l smallGrid
```

O argumento `-n` indica a quantidade de episódios (partidas) que o algoritmo vai executar ao todo, e o argumento `-x` indica quantos desses episódios serão apenas de treinamento (a saída não será mostrada no terminal). Nesse caso, o agente irá treinar por 2000 episódios e então mostrar a execução dos últimos 10.

Durante o treinamento, você verá uma saída a cada 100 partidas com estatísticas sobre o desempenho do Pac-Man. É esperado que leve de 1000 a 1400 jogos para que o Pac-Man comece a obter uma recompensa média positiva.

4 Relatório

Após o desenvolvimento dos experimentos, você deverá analisar seus algoritmos e redigir um relatório claro e sucinto (de 2 a 3 páginas, no máximo), fazendo uma análise comparativa de desempenho dos algoritmos implementados com auxílio de gráficos e tabelas.

4.1 Análise dos algoritmos no Gridworld

Para analisar o comportamento do agente com o algoritmo de Iteração de Valor, gere resultados da execução no mapa `DiscountGrid` para os 5 casos de testes da Tabela 1.

	MDP 1	MDP 2	MDP 3	MDP 4	MDP 5
Fator de desconto (<code>--discount</code>)	0.1	0.5	1	0.9	1
Chance de erro (<code>--noise</code>)	0	0.2	0	0.2	0.2
Recompensa por ação (<code>--livingReward</code>)	0	-2	-1	0	10

Tabela 1: Valores dos parâmetros dos cinco casos de teste do mapa `DiscountGrid`.

Compile em uma tabela as *recompensas acumuladas médias* obtidas em cada caso após 10 episódios (com o argumento `-k`). Discuta as diferenças entre as políticas obtidas e a relação delas com os parâmetros de cada caso de teste.

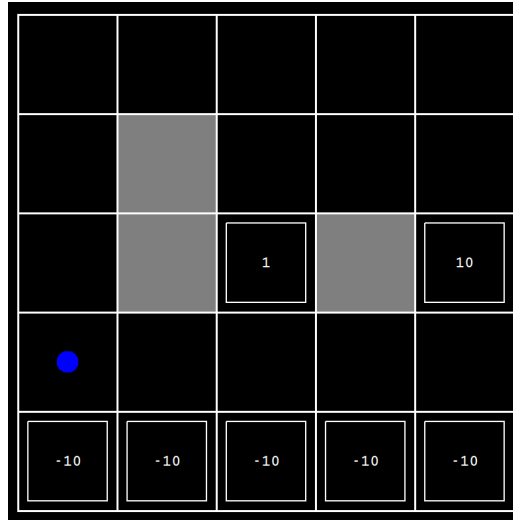


Figura 2: Mapa do *DiscountGrid*, a ser usado para avaliar as políticas geradas pela Iteração de Valor, com os diferentes parâmetros descritos na tabela 1.

Para analisar o Q-Learning e compará-lo com a Iteração de Valor, teste ambos os algoritmos no cenário *CliffGrid* (figura 3) com 100 episódios de treinamento. Discuta a diferença entre os valores $V(s)$ finais encontrados pelos dois algoritmos, mostrados na interface ao final do treinamento.

Dica: Ao final do treinamento do Q-Learning, será mostrada uma tela com os valores Q de cada ação. Aperte uma tecla para mostrar a tela com os valores V dos estados.

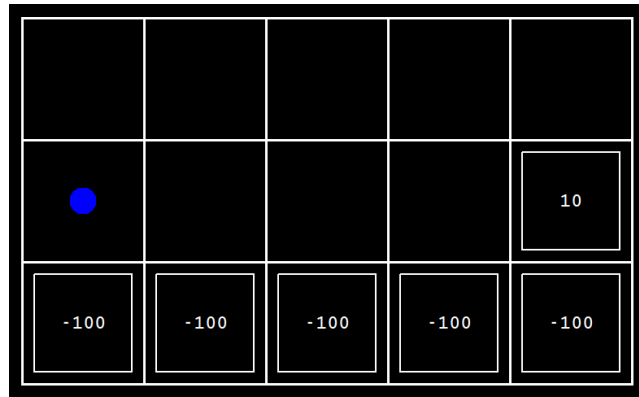


Figura 3: Mapa do *CliffGrid* que deve ser usado para analisar e comparar os algoritmos Q-Learning e Iteração de Valor (não há necessidade de mudar os parâmetros para essa análise).

4.2 Análise do Q-Learning no Pac-Man

Analise o algoritmo Q-Learning no Pac-Man em dois cenários, `smallGrid` (Figura 4) e `tinyCorners` (Figura 5), com 2000 episódios de treinamento. O simulador imprime a média dos *scores* finais dos últimos 100 episódios. Analise a convergência desses valores para os dois cenários usando dois valores de α diferentes: 0.2 (padrão) e 1. Compare também *score* final médio de 100 episódios *depois* da

convergência do algoritmo com o *score* final médio de 100 episódios obtido pelo algoritmo `GreedyAgent`. Para testar o `GreedyAgent`, use os comandos:

```
$ python pacman.py -p GreedyAgent -l smallGrid -n 100
$ python pacman.py -p GreedyAgent -l tinyCorners -n 100
```

Dica: Lembre-se que você usar adicionar a flag `-q` ao comando para pular a simulação gráfica do jogo.



Figura 4: Mapa do *smallGrid*.

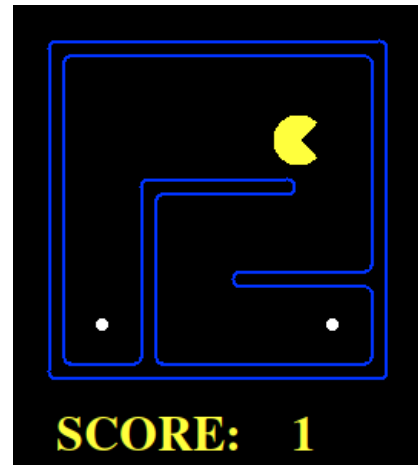


Figura 5: Mapa do *tinyCorners*.

5 Entrega

Você deve entregar **três arquivos** separados:

- (1) O arquivo `valueIterationAgents.py` modificado com a sua implementação da Iteração de Valor;
- (2) O arquivo `qlearningAgents.py` modificado com a sua implementação do Q-Learning;
- (3) Um arquivo chamado `rel-EP4-NUSP-NOME-COMPLETO.pdf` em formato PDF contendo o relatório com as análises e comparações dos dois algoritmos (máximo de 3 páginas).

Não esqueça de identificar cada arquivo com seu nome e número USP! Faça isso tanto no código (coloque um cabeçalho em forma de comentário) quanto no relatório (coloque em destaque no título).

6 Critério de avaliação

O critério de avaliação atribuirá pesos iguais à parte prática e ao relatório. A avaliação da parte prática dependerá parcialmente dos resultados dos testes automatizados do `autograder.py`. Dessa forma você terá como avaliar por si só parte da nota que receberá para a parte prática. Para rodar os testes automatizados pelo `autograder`, execute o seguinte comando:

```
$ python autograder.py
```

A saída do autograder é a pontuação obtida em cada questão incluindo detalhes dos testes que deram errado (caso não tenha passado).

Com relação ao relatório, avaliaremos principalmente sua forma de interpretar comparativamente o desempenho de cada algoritmo em um mesmo cenário. Não é necessário detalhar a estratégia de cada um ou qual estrutura de dados você utilizou, mas deve ficar claro que você compreendeu os resultados obtidos conforme esperado dadas as características teóricas de cada algoritmo.